

实验八 数组程序设计（二）二维数组与字符串

一、 实验目的：

1. 熟练掌握二维数组编写程序；
2. 理解字符数组与字符串，掌握字符串的存储和操作编程。

二、 实验要求：

1. 用 C 语言实现源程序的编写；
2. 源程序应书写规范，适当添加注释；
3. 实验课前完成实验内容源程序的初稿；实验课对源程序进行编辑、调试、运行；实验课后尽快完成实验报告，并按规定时间上交实验报告。

三、 实验内容及结果：

1. 判断上三角矩阵。输入一个正整数 $n(1 \leq n \leq 6)$ 和 n 阶方阵 a 中的元素，如果 a 是上三角矩阵，输出“YES”，否则，输出“NO”。上三角矩阵指主对角线以下的元素都为 0 的矩阵，主对角线为从矩阵的左上角至右下角的连线。试编写相应程序。

```
#include <stdio.h>

_Bool judgeUpTriangle(int n, int a[n][n]){

    for(int i = 1; i < n; i++)
        for(int j = 0; j < i; j++)
            if(a[i][j] != 0)
                return 0;

    return 1; //全部元素都为 0，是上三角矩阵
}

/*
4
1 1 1 1
0 1 1 1
0 0 1 1
0 0 0 1
*/
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);
    int a[n][n];
```

```

        for(int i = 0; i < n; i++)
            for(int j = 0; j < n; j++)
                scanf("%d", &a[i][j]);

        if(judgeUpTriangle(n, a))
            printf("YES\n");
        else
            printf("NO\n");
    }

```

测试结果:

```

4
1 1 1 1
0 1 1 1
0 0 1 1
0 0 0 1
YES

```

2. 求矩阵各行元素之和。输入 2 个正整数 m 和 n ($1 \leq m \leq 6, 1 \leq n \leq 6$)，然后输入矩阵 a (m 行 n 列) 中的元素，分别求出各行元素之和，并输出。试编写相应程序。

```

#include <stdio.h>

void getRowSum(int n, int m, int a[n][m]){
    for(int i = 0; i < n; i++){
        int sum = 0;

        printf("第%d 行的元素和是:", i + 1);

        for(int j = 0; j < m; j++)
            sum += a[i][j];
        printf("%d\n", sum);
    }
}

/*
3 4
1 3 -1 1
8 3 9 9

```

```

-5 -3 2 6
*/
int main(int argc, char *argv[]) {
    int n, m;
    scanf("%d %d", &n, &m);
    int a[n][m];

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &a[i][j]);

    getRowSum(n, m, a);
}

```

测试结果:

```

3 4
1 3 -1 1
8 3 9 9
-5 -3 2 6
第1行的元素和是:4
第2行的元素和是:29
第3行的元素和是:0

```

3. 找鞍点。输入 1 个正整数 n ($1 \leq n \leq 6$) 和 n 阶方阵 a 中的元素, 假设方阵 a 最多有 1 个鞍点, 如果找到 a 的鞍点, 就输出其下标, 否则, 输出 "NO"。鞍点的元素值在该行上最大, 在该列上最小。试编写相应程序。

```

#include <stdio.h>

int getMin(int column, int row, int a[row][row]){ //注意传入
数组大小的细节, 否则传入元素出错
    int min = a[0][column];

    for(int i = 1; i < row; i++){ //求当前列的最小值
        printf("%d\n", a[i][column]);
        if(a[i][column] < min) //注意对列的遍历
            min = a[i][column];
    }
    return min;
}

```

```

}

int getMax(int n, int a[n]){
    int max = a[0];
    int index = 0;
    for(int i = 1; i < n; i++)
        if(a[i] > max){
            max = a[i];
            index = i;
        }

    return index; //返回当前最大值的索引（地址）
}

void find(int n, int a[n][n]){
    int flag = 1;

    for(int i = 0; i < n; i++){//遍历n行

        int index = getMax(n, a[i]);
        int minOfColumn = getMin(index , n, a);
        if(a[i][index] == minOfColumn){
            printf("Matrix's saddle point is : %d\n",
minOfColumn);
            flag = 0;
        }
    }
    if(flag)
        printf("NO!");
}

/*
4
1 5 9 2
-3 2 10 9
2 0 8 1
4 2 7 1
*/
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);
    int a[n][n];

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
}

```

```

        find(n, a);
//      for(int i = 0; i < n; i++){
//          for(int j = 0; j < n; j++){
//              printf("%d ", a[i][j]);
//          }
//          printf("\n");
//      }
}

```

测试结果:

```

4
1 5 9 2
-3 2 10 9
2 0 8 1
4 2 7 1
Matrix's saddle point is : 7

```

4. 字符串替换。输入一个以回车结束的字符串（少于 80 个字符），将其中的大写字母用下面列出的对应大写字母替换，其余字符不变，输出替换后的字符串。试编写相应程序。

A→Z, B→Y, C→X...X→C, Y→B, Z→A

```

#include <stdio.h>

void replaeChar(char s[80]){
    //遍历字符串
    for(int i = 0; i < 80 && s[i] != '\0'; i++){
        //A -> 65 .. Z -> 90
        if(s[i] >= 'A' && s[i] <= 'Z'){
            //总结规律啦~
            s[i] = 90 - (s[i] - 65);
        }
    }

    printf("替换后的字符串:\n");
    for(int i = 0; i < 80 && s[i] != '\0'; i++){
        printf("%c ", s[i]);
    }
}

```

```
}  
//ABCDEFGHIJKLMNOPQRSTUVWXYZ  
int main(int argc, char *argv[]) {  
    char str[80];  
    scanf("%s", str);  
  
    replaeChar(str);  
}
```

测试结果:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

替换后的字符串:

Z Y X W V U T S R Q P O N M L K J I H G F E D C B A

四、 实验小结:

1. 经过二维数组和矩阵的联系，熟练掌握了如何利用二维数组存储数据，以及对数据相应的操作，和对应项目需求进行的一系列判断。
2. 经过对字符串的对称变换，深刻理解了字符数组与字符串的联系，即字符串的每个字符都作为独立的元素存储于字符数组之中，其中最后一个元素以‘\0’作为结束标记，要记住它占一个空间，最终熟练掌握了字符串的存储和相应所需的操作。

五、 实验成绩: